Pattern Matching with switch

A `switch` expression or statement transfers control to one of several statements or expressions, depending on the value of its selector expression. The selector expression can be any reference or primitive type.

Also, `case` labels can have patterns. Consequently, a `switch` expression or statement can test whether its selector expression matches a pattern, which offers more flexibility and expressiveness compared to testing whether its selector expression is exactly equal to a constant.

For background information about pattern matching for `switch` expressions and statements, see JEP 441.

Consider the following code that calculates the perimeter of certain shapes from the section Pattern Matching with instanceof:

COPY

```
interface Shape { }
record Rectangle(double length, double width) implements Shape { }
record Circle(double radius) implements Shape { }
// ...

public static double getPerimeter(Shape s) throws IllegalArgumentException {
    if (s instanceof Rectangle r) {
        return 2 * r.length() + 2 * r.width();
    } else if (s instanceof Circle c) {
        return 2 * c.radius() * Math.PI;
    } else {
        throw new IllegalArgumentException("Unrecognized shape");
    }
}
```

You can rewrite this code to use a pattern `switch` expression as follows:

COPY

```
public static double getPerimeter(Shape s) throws IllegalArgumentException {
    return switch (s) {
        case Rectangle r ->
            2 * r.length() + 2 * r.width();
        case Circle c ->
            2 * c.radius() * Math.PI;
        default ->
            throw new IllegalArgumentException("Unrecognized shape");
    };
}
```

The following example uses a `switch` statement instead of a `switch` expression:

COPY

```
public static double getPerimeter(Shape s) throws IllegalArgumentException {
    switch (s) {
        case Rectangle r:
            return 2 * r.length() + 2 * r.width();
        case Circle c:
            return 2 * c.radius() * Math.PI;
        default:
            throw new IllegalArgumentException("Unrecognized shape");
    }
}
```

Topics

- When Clauses
- Pattern Label Dominance
- Scope of Pattern Variables and switch
- Null case Labels

When Clauses

You can add a Boolean expression right after a pattern label with a `when` clause. This is called a *guarded pattern label*. The Boolean expression in the `when` clause is called a *guard*. A value matches a guarded pattern label if it matches the pattern and, if so, the guard also evaluates to true. Consider the following example:

COPY

```
static void test(Object obj) {
    switch (obj) {
        case String s:
            if (s.length() == 1) {
                System.out.println("Short: " + s);
            } else {
                System.out.println(s);
            }
            break;
        default:
            System.out.println("Not a string");
    }
}
```

You can move the Boolean expression `s.length == 1` right after the `case` label with a `when` clause:

COPY

```
static void test(Object obj) {
    switch (obj) {
        case String s when s.length() == 1 -> System.out.println("Short: " + s);
        case String s                      -> System.out.println(s);
        default                            -> System.out.println("Not a string");
    }
}
```

The first pattern label (which is a guarded pattern label) matches if `obj` is both a `String` and of length 1. The second patten label matches if `obj` is a `String` of a different length.

A guarded patten label has the form `p when e` where *p* is a pattern and `e` is a Boolean expression. The scope of any pattern variable declared in *p* includes *e*.

The following example demonstrates the use of a `when` clause with a record class:

```
record R(int x) { }
// ...
static void testSwitch(R r) {
    switch(r) {
        case R(int x) when x >= 5 -> System.out.println(x + " => 5");
        default -> System.out.println(r.x + " < 5");
    }
}
```

The guard `case R(int x) when x >= 5` contains a type pattern `int x` nested in a record pattern. See Record Patterns.

The following example also demonstrates `when` clauses. It uses a primitive type, `double`, for its `switch` expression's selector expression:

```
String doubleToRating(double rating) {
    return switch(rating) {
        case 0d -> "0 stars";
        case double d when d > 0d && d < 2.5d
            -> d + " is not good";
        case double d when d >= 2.5f && d < 5d
            -> d + " is better";
        case 5d -> "5 stars";
        default -> "Invalid rating";
    };
}
```

The guard `case double d when d > 0d && d < 2.5d` contains a type pattern `double d` that's *not* nested in a record pattern. This is called a top-level pattern.

> **Note**
>
> Guards restricting values of *primitive types* coming from a top-level pattern is a preview feature. A preview feature is a feature whose design, specification, and implementation are complete, but is not permanent. A preview feature may exist in a different form or not at all in future Java SE releases. To compile and run code that contains preview features, you must specify additional command-line options. See Preview Language and VM Features.
>
> See JEP 507: Primitive Types in Patterns, instanceof, and switch (Third Preview) for additional information.

The following example uses a `long` for its selector expression:

```
void bigNumbers(long v) {
    switch (v) {
        case long x when x < 1_000_000L ->
            System.out.println("Less than a million");
        case long x when x < 1_000_000_000L ->
            System.out.println("Less than a billion");
        case long x when x < 1_000_000_000_000L ->
            System.out.println("Less than a trillion");
        case long x when x < 1_000_000_000_000_000L ->
            System.out.println("Less than a quadrillion");
        default -> System.out.println("At least a quadrillion");
    }
}
```

Pattern Label Dominance

It's possible that many pattern labels could match the value of the selector expression. To help predictability, the labels are tested in the order that they appear in the `switch` block. In addition, the compiler raises an error if a pattern label can never match because a preceding one will always match first. The following example results in a compile-time error:

COPY

```
static void error(Object obj) {
    switch(obj) {
        case CharSequence cs ->
            System.out.println("A sequence of length " + cs.length());
        // error: this case label is dominated by a preceding case label
        case String s ->
            System.out.println("A string: " + s);
        default -> { break; }
    }
}
```

The first pattern label `case CharSequence cs` *dominates* the second pattern label `case String s` because every value that matches the pattern `String s` also matches the pattern `CharSequence cs` but not the other way around. It's because `String` is a subtype of `CharSequence`.

A pattern label can dominate a constant label. These examples cause compile-time errors:

COPY

```
static void error2(Integer value) {
    switch(value) {
        case Integer i ->
            System.out.println("Integer: " + i);
        // Compile-time errors for both cases -1 and 1:
        // this case label is dominated by a preceding case label
        case -1, 1 ->
            System.out.println("The number 42");
        default -> { break; }
    }
}
// ...

enum Color { RED, GREEN, BLUE; }
// ...

static void error3(Color value) {
    switch(value) {
        case Color c ->
            System.out.println("Color: " + c);
        // error: this case label is dominated by a preceding case label
        case RED ->
            System.out.println("The color red");
    }
}
```

Note

Guarded pattern labels don't dominate constant labels. For example:

```
static void testInteger(Integer value) {
    switch(value) {
        case Integer i when i > 0 ->
            System.out.println("Positive integer");
        case 1 ->
            System.out.println("Value is 1");
        case -1 ->
            System.out.println("Value is -1");
        case Integer i ->
            System.out.println("An integer");
    }
}
```

Although the value `1` matches both the guarded pattern label `case Integer i when i > 0` and the constant label `case 1`, the guarded pattern label doesn't dominate the constant label. Guarded patterns aren't checked for dominance because they're generally undecidable. Consequently, you should order your case labels so that constant labels appear first, followed by guarded pattern labels, and then followed by nonguarded pattern labels:

```
static void testIntegerBetter(Integer value) {
    switch(value) {
        case 1 ->
            System.out.println("Value is 1");
        case -1 ->
            System.out.println("Value is -1");
        case Integer i when i > 0 ->
            System.out.println("Positive integer");
        case Integer i ->
            System.out.println("An integer");
    }
}
```

Scope of Pattern Variables and switch

In a `switch` expression or statement, the scope of a pattern variable declared in a `case` label includes the following:

- The `when` clause of the `case` label:

  COPY

  ```
  static void test(Object obj) {
      switch (obj) {
          case Character c when c.charValue() == 7:
              System.out.println("Ding!");
              break;
          default:
              break;
          }
      }
  }
  ```

  The scope of pattern variable `c` includes the `when` clause of the `case` label that contains the declaration of `c`.

- The expression, block, or `throw` statement that appears to the right of the arrow of the `case` label:

  COPY

  ```
  static void test(Object obj) {
      switch (obj) {
          case Character c -> {
              if (c.charValue() == 7) {
                  System.out.println("Ding!");
              }
              System.out.println("Character, value " + c.charValue());
          }
          case Integer i ->
              System.out.println("Integer: " + i);
          default -> {
              break;
          }
      }
  }
  ```

  The scope of pattern variable `c` includes the block to the right of `case Character c ->`. The scope of pattern variable `i` includes the `println` statement to the right of `case Integer i ->`.

- The `switch`-labeled statement group of a `case` label:

```
static void test(Object obj) {
    switch (obj) {
        case Character c:
            if (c.charValue() == 7) {
                System.out.print("Ding ");
            }
            if (c.charValue() == 9) {
                System.out.print("Tab ");
            }
            System.out.println("character, value " + c.charValue());
        default:
            // You cannot use the pattern variable c here:
            break;
    }
}
```

The scope of pattern variable `c` includes the `case Character c` statement group: the two `if` statements and the `println` statement that follows them. The scope doesn't include the `default` statement group even though the `switch` statement can execute the `case Character c` statement group, fall through the `default` label, and then execute the `default` statement group.

> **Note**
>
> You will get a compile-time error if it's possible to fall through a case label that declares a pattern variable. The following example doesn't compile:

```
static void test(Object obj) {
    switch (obj) {
        case Character c:
            if (c.charValue() == 7) {
                System.out.print("Ding ");
            }
            if (c.charValue() == 9) {
                System.out.print("Tab ");
            }
            System.out.println("character");
        case Integer i: // Compile-time error
            System.out.println("An integer " + i);
        default:
            System.out.println("Neither character nor integer");
    }
}
```

If this code were allowed, and the value of the selector expression, `obj`, were a `Character`, then the `switch` statement can execute the `case Character c` statement group and then fall through the `case Integer i` label, where the pattern variable `i` would have not been initialized.

See [Scope of Pattern Variables and instanceof]() for more examples of where you can use a pattern variable.

Null case Labels

`switch` expressions and `switch` statements used to throw a `NullPointerException` if the value of the selector expression is `null`. Currently, to add more flexibility, a `null` case label is available:

```
static void test(Object obj) {
    switch (obj) {
        case null     -> System.out.println("null!");
        case String s -> System.out.println("String");
        default       -> System.out.println("Something else");
    }
}
```

This example prints `null!` when `obj` is `null` instead of throwing a `NullPointerException`.

You may not combine a `null` case label with anything but a `default` case label. The following generates a compiler error:

```
static void testStringOrNull(Object obj) {
    switch (obj) {
        // error: invalid case label combination
        case null, String s -> System.out.println("String: " + s);
        default             -> System.out.println("Something else");
    }
}
```

However, the following compiles:

```
static void testStringOrNull(Object obj) {
    switch (obj) {
        case String s       -> System.out.println("String: " + s);
        case null, default  -> System.out.println("null or not a string");
    }
}
```

If a selector expression evaluates to `null` and the `switch` block does not have `null` case label, then a `NullPointerException` is thrown as normal. Consider the following `switch` statement:

```
String s = null;
switch (s) {
    case Object obj -> System.out.println("This doesn't match null");
    // No null label; NullPointerException is thrown
    //     if s is null
}
```

Although the pattern label `case Object obj` matches objects of type `String`, this example throws a `NullPointerException`. The selector expression evaluates to `null`, and the `switch` expression doesn't contain a `null` case label.